

# Deep Q-Networks: relatório experimental

Dupla: Gustavo Novack (novack@discente.ufg.br) e Davi de Souza Andrade (davisouza@discente.ufg.br). Fork com o `algorithms/dqn.py` completo: <https://github.com/Novack-IA/t-zero-learning>

Ambiente: CartPole-v1, 500.000 passos por run, CPU. Duas seeds (1 e 2) em toda configuração de toda varredura. As curvas mostram a média das duas seeds e a faixa sombreada cobre o intervalo entre elas; `td_loss` e `q_values` aparecem suavizados por média móvel de 40 pontos (1 ponto = 100 passos). As métricas foram registradas localmente em CSV, com o mesmo conjunto de séries que o harness envia ao wandb, e os gráficos foram gerados por `scripts/dqn_assignment/make_figures.py`; os scripts de varredura e de plotagem estão no fork.

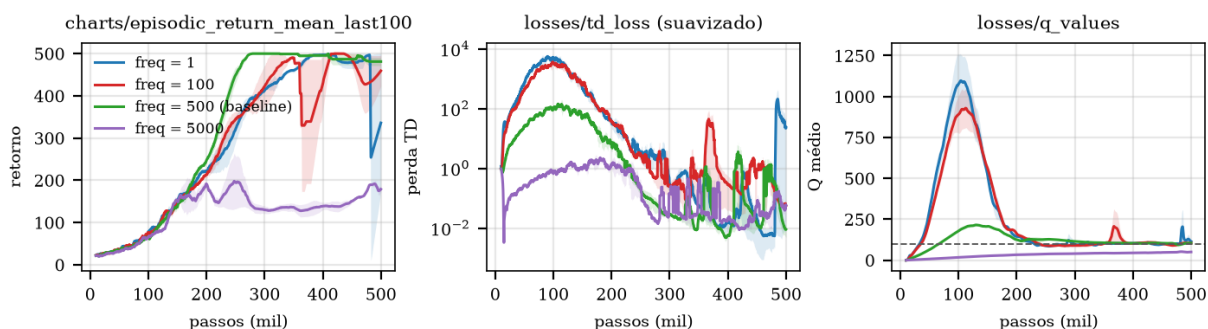
## Implementação (Partes 1 e 2)

Os três blocos foram completados em `algorithms/dqn.py`: `ReplayBuffer.add` escreve na posição `pos` e avança de forma circular (`pos = (pos + 1) % capacidade`), mantendo `size` saturado na capacidade; `ReplayBuffer.sample` sorteia índices em `[0, size)`, e não em `[0, capacidade)`, para nunca tocar em slots vazios; `compute_td_targets` devolve  $r + \gamma \max_a Q_{alvo}(s', a) (1 - done)$ , com `rewards` e `done`s achatados de `(B,1)` para `(B,)` antes da soma, pois sem isso o broadcasting produz um alvo `(B,B)` silenciosamente errado. Os sete testes de `tests/test_dqn.py` passam. O baseline (configuração original do `configs/dqn_cartpole.yml`) atinge retorno 500 e avaliação gulosa final de  $500,0 \pm 0,0$  nas duas seeds.

## Q1. Frequência de sincronização da target network

**Previsão.** Com `freq=1` o alvo deixa de ser fixo e passa a se mover junto com a rede que está sendo ajustada; eu esperava uma realimentação do erro sobre si mesmo, `q_values` crescendo sem controle e retorno instável ou em colapso. Com `freq=5000` o alvo fica quase congelado, então eu esperava aprendizado lento, `q_values` em degraus e picos de `td_loss` a cada sincronização. O valor intermediário deveria ser o melhor.

**Varredura.** `dqn.target_network_frequency` em 1, 100, 500 (baseline) e 5000, duas seeds cada, oito runs.



Q1. Da esquerda para a direita: retorno médio dos últimos 100 episódios, perda de TD (escala logarítmica) e Q médio predito. A linha tracejada de referência para o Q correto neste ambiente é  $1/(1-0,99) = 100$ .

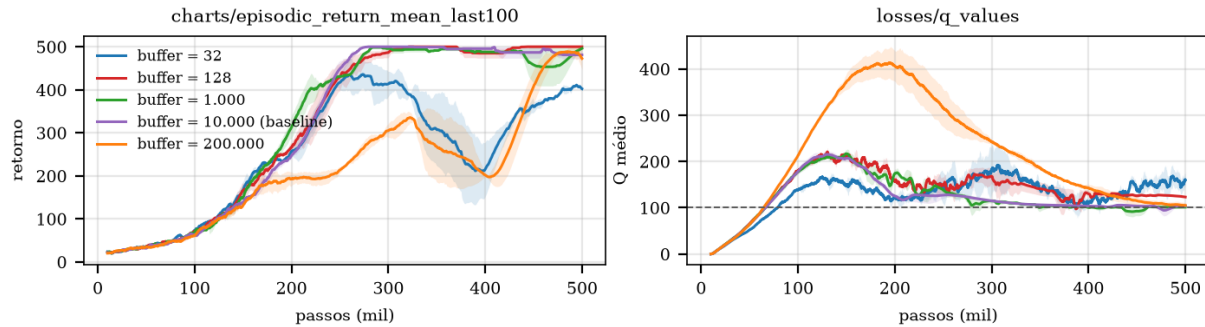
**Explicação.** O papel da target network é congelar o alvo do bootstrap para que, entre duas sincronizações, o passo de aprendizado seja uma regressão comum contra rótulos fixos. Sem esse congelamento o alvo  $r + \gamma \max_a Q(s', a)$  é função dos mesmos pesos que estão sendo atualizados, e cada passo que aumenta  $Q(s,a)$  também aumenta o próprio alvo. O gráfico de `q_values` mostra exatamente essa realimentação: com `freq=1` e `freq=100` o Q médio sobe até cerca de 1100 e 950 por volta de 110 mil passos, enquanto o valor verdadeiro em CartPole com  $\gamma = 0,99$  e recompensa 1 por passo é limitado por  $1/(1-0,99) = 100$ . É uma superestimação de mais de dez vezes, isto é, divergência, e não erro de estimativa. O baseline com `freq=500` tem um excesso muito menor (pico em torno de 210) e converge para perto de 100, que é o valor correto. A previsão se confirmou quanto à divergência dos valores, mas errou quanto ao desfecho: apesar da divergência, `freq=1` e `freq=100` ainda chegam a 500 de retorno, porque o CartPole é tolerante e a política gulosa depende só da ordem relativa entre as duas ações, não da escala de Q. O custo aparece na estabilidade, com a queda de `freq=1` perto de 490 mil passos e avaliação gulosa de 500,0 e 443,0 nas duas seeds, contra 500,0 e 500,0 do baseline.

O outro extremo responde à pergunta sobre o `td_loss` parecer bem enquanto o retorno degrada. `freq=5000` tem a menor perda de TD de toda a varredura, abaixo de 1 na maior parte do treino, e mesmo assim é a pior configuração: retorno estacionado em torno de 150 a 180 e avaliação gulosa de  $186,0 \pm 4,2$  e  $196,0 \pm 12,6$ . A razão é que `td_loss` mede a distância entre Q e o alvo, não entre Q e o valor verdadeiro. Com o alvo quase congelado a rede resolve muito bem uma regressão contra rótulos obsoletos, e perda baixa passa a significar "eu reproduzo meu alvo", não "meu Q está certo". Por isso a curva de retorno precisa ser lida junto com `q_values`: lá se vê que `freq=5000` nunca chega perto de 100, subindo quase linearmente e parando abaixo de 60. Cada cópia de pesos propaga a informação de recompensa por aproximadamente mais um passo de bootstrap, e 500 mil passos divididos por 5000 dão apenas 100 sincronizações no treino inteiro, o que limita o horizonte efetivo que a rede consegue enxergar. O aprendizado não está instável, está truncado.

## Q2. Tamanho do replay buffer

**Previsão.** Com buffer minúsculo o minibatch vem de uma janela de poucos passos quase consecutivos, portanto amostras muito correlacionadas e esquecimento do que a política antiga visitava; eu esperava retorno baixo e oscilante. Buffer grande deveria ser igual ou levemente melhor que o baseline.

**Varredura.** `dqn.buffer_size` em 32, 128, 1.000, 10.000 (baseline) e 200.000, duas seeds cada, dez runs.



Q2. Retorno médio dos últimos 100 episódios e Q médio predito, para cinco tamanhos de buffer.

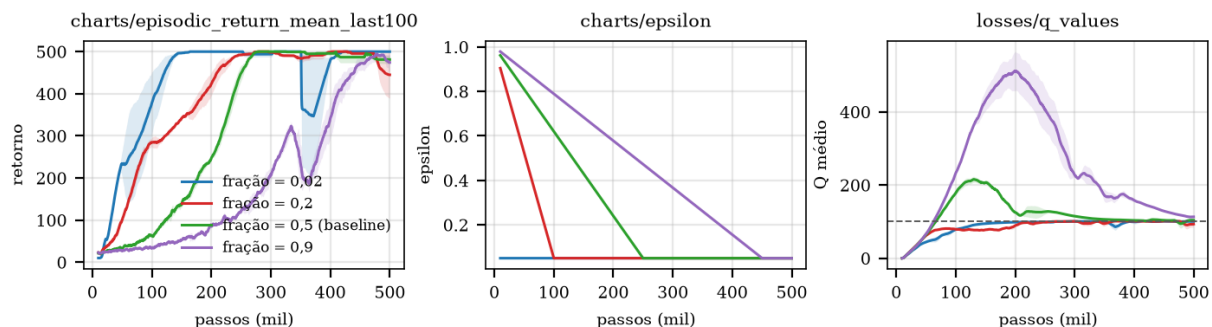
**Explicação.** Um buffer pequeno demais causa dois problemas distintos. O primeiro é de correlação dentro do minibatch: com capacidade 32 e `batch_size=128`, os 128 exemplos são sorteados com reposição de apenas 32 transições temporalmente adjacentes do mesmo trecho de episódio, então o gradiente é calculado sobre um punhado de pontos quase idênticos. A hipótese de amostras aproximadamente independentes que justifica o passo de SGD deixa de valer e cada atualização vira um passo grande e enviesado na direção da situação momentânea. O segundo é de cobertura: o buffer só guarda os últimos 32 passos, de modo que os estados visitados pelas políticas anteriores, em particular os estados de quase-falha com o poste já inclinado, são sobrescritos antes de poderem ser revistos. A rede esquece como avaliá-los e perde o sinal corretivo justamente onde ele importa. Os dois efeitos aparecem nos gráficos: a curva de `buffer = 32` sobe junto com as demais até cerca de 250 mil passos, depois descola, cai e estaciona perto de 400, e sua avaliação gulosa final é  $276,3 \pm 4,2$  e  $251,1 \pm 9,2$ , ou seja, a política gulosa é de fato pior, e não apenas ruidosa. No gráfico de `q_values` a mesma configuração se estabiliza entre 130 e 150 na segunda metade do treino, contra cerca de 102 do baseline, uma superestimção persistente de perto de 40 por cento que nunca é corrigida porque as transições capazes de corrigi-la já foram sobrescritas.

A previsão se confirmou no extremo, mas superestimou a fragilidade da faixa intermediária: `buffer = 128` e `buffer = 1.000` acompanham o baseline e também terminam em 500,0 na avaliação gulosa. O CartPole é permissivo nesse aspecto, porque a recompensa é 1 em todo estado não terminal e a função de valor é suave, então mesmo uma janela curta ainda contém exemplos representativos. A previsão errou por completo no outro extremo. `buffer = 200.000` é a configuração mais lenta e mais instável do experimento: precisa de cerca de 465 mil passos para alcançar retorno 475, contra 260 mil do baseline, o desvio-padrão do retorno na última fase é de 118 contra 10 do baseline, e o `q_values` dispara até 410. Com 500 mil passos no total, um buffer de 200 mil ainda contém 40 por cento de todo o histórico, boa parte dele gerado pela política quase aleatória do início; os minibatches ficam dominados por dados que a política atual nunca produziria, e a segunda seed termina com avaliação gulosa de  $298,0 \pm 137,5$ . O tamanho do buffer é, portanto, um compromisso de dois lados: pequeno demais correlaciona e esquece, grande demais dilui o minibatch com dados obsoletos.

### Q3. Fração de exploração (extra)

**Previsão.** Com fração 0,02 o epsilon cai para 0,05 quase de imediato, antes de o buffer ter dados variados, e eu esperava política gulosa cedo demais e retorno preso num valor baixo. Com 0,9 o agente passa quase todo o treino agindo ao acaso em boa parte dos passos, então o retorno durante o treino ficaria limitado pelo próprio ruído da exploração.

**Varredura.** `dqn.exploration_fraction` em 0,02, 0,2, 0,5 (baseline) e 0,9, duas seeds cada, oito runs. **Gráficos escolhidos:** retorno, `charts/epsilon` e `losses/q_values`. O epsilon é obrigatório aqui porque o retorno registrado durante o treino é o da política de comportamento, não da gulosa; sem saber quanta ação aleatória estava em vigor em cada instante, a curva de retorno não é interpretável e a comparação entre configurações não é justa. O `q_values` entra para separar duas explicações que a curva de retorno sozinha confunde: "explorou menos, logo aprendeu pior" e "explorou menos, logo o retorno medido foi menos penalizado por ações aleatórias".



Q3. Retorno médio dos últimos 100 episódios, cronograma de epsilon efetivamente aplicado e Q médio predito.

**Explicação.** Na curva de retorno, `fração = 0,02` chega a 500 por volta de 150 mil passos e `fração = 0,9` só por volta de 450 mil, o que sugeriria uma diferença enorme de velocidade de aprendizado. O gráfico de epsilon mostra que boa parte disso é artefato de medição: em 250 mil passos a configuração 0,9 ainda age ao acaso em cerca de 45 por cento dos passos e, em CartPole, quase toda ação aleatória num estado avançado encerra o episódio. O retorno medido fica mecanicamente limitado por isso, por melhor que a rede esteja. A avaliação gulosa final resolve a ambiguidade e mostra que as quatro configurações aprenderam uma política boa: 500,0/500,0 para 0,02, 498,0/500,0 para 0,2, 500,0/500,0 para o baseline e 500,0/480,0 para 0,9. A diferença real de velocidade existe, mas é bem menor do que a curva de treino faz parecer.

O `q_values` mostra o mecanismo que o retorno esconde. A configuração 0,9 sobe até cerca de 500, cinco vezes o limite verdadeiro de 100, enquanto 0,02 e 0,2 permanecem próximas de 100 o treino inteiro. Uma fase longa de epsilon alto mantém o buffer cheio de transições de uma política quase aleatória, e o `max` sobre ações no alvo de TD é tomado sobre estados cujos valores estão mal estimados; como o máximo de estimativas ruidosas é enviesado para cima, o viés se acumula enquanto os dados continuam fora da política. Quando o epsilon finalmente decai, os dados voltam a ser da política corrente e as estimativas são puxadas de volta para perto de 100, o que é exatamente o formato de corcova da curva laranja. A previsão sobre 0,9 se confirmou quanto ao retorno de treino limitado, mas eu não previ a superestimção; e a previsão sobre 0,02 estava errada, pois foi a configuração mais rápida. O motivo é que o epsilon nunca chega a zero, já que `end_e = 0,05`, e os 10.000 passos iniciais de `learning_starts` são inteiramente aleatórios; num problema que não exige exploração profunda, 5 por cento de ações aleatórias já dão cobertura suficiente, e encurtar o cronograma apenas remove ruído.